

data_into_gold.sql

Location: scripts/gold/data_into_gold.sql | Target Layer: Gold Layer (Conformed Analytical Star Schema Zone)

```
-- =====
-- ETL Script: Load Gold Tables (Silver -> Gold Star Schema)
-- Database: cadetos_gold (Reads from cadetos_silver)
-- Project : CadetOS Data Warehouse
-- =====
-- Purpose:
--     Transforms and loads clean data from cadetos_silver into the star schema
--     in cadetos_gold for business intelligence and dashboards.
--
-- Execution Order:
--     1. Clear tables (preserving schemas).
--     2. Populate reference dimensions (dim_rank, dim_certificate, dim_camp).
--     3. Generate calendar dimension using recursive CTE (dim_date).
--     4. Populate Cadet Master dimension (dim_cadet).
--     5. Populate Fact Tables (fct_attendance, fct_camp_participation, etc.).
--     6. Run updates for pre-calculated aggregates (total camps & parades).
--     7. Populate Aggregate/Summary Tables (agg_cadet_annual_metrics, agg_unit_summary).
-- =====

USE cadetos_gold;

-- =====
-- STEP 1: TRUNCATE/DELETE PREVIOUS DATA (Safely with constraints disabled)
-- =====

SET FOREIGN_KEY_CHECKS = 0;
SET SQL_SAFE_UPDATES = 0;

DELETE FROM agg_cadet_annual_metrics;
ALTER TABLE agg_cadet_annual_metrics AUTO_INCREMENT = 1;

DELETE FROM agg_unit_summary;
ALTER TABLE agg_unit_summary AUTO_INCREMENT = 1;

DELETE FROM fct_attendance;
ALTER TABLE fct_attendance AUTO_INCREMENT = 1;

DELETE FROM fct_camp_participation;
ALTER TABLE fct_camp_participation AUTO_INCREMENT = 1;

DELETE FROM fct_rank_assignment;
ALTER TABLE fct_rank_assignment AUTO_INCREMENT = 1;

DELETE FROM fct_certificate_achievement;
ALTER TABLE fct_certificate_achievement AUTO_INCREMENT = 1;

DELETE FROM dim_cadet;
ALTER TABLE dim_cadet AUTO_INCREMENT = 1;

DELETE FROM dim_rank;
DELETE FROM dim_certificate;
DELETE FROM dim_camp;
DELETE FROM dim_date;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

```
-- =====
-- STEP 2: POPULATE REFERENCE DIMENSIONS
-- =====
```

```
-- 2.1 Load Ranks Reference (dim_rank)
```

```
INSERT INTO dim_rank (rank_id, rank_name, rank_level, rank_description)
VALUES
```

```
    (1, 'SUO', 1, 'Senior Under Officer'),
    (2, 'JUO', 2, 'Junior Under Officer'),
    (3, 'CQMS', 3, 'Company Quartermaster Sergeant'),
    (4, 'CSM', 4, 'Company Sergeant Major'),
    (5, 'SGT', 5, 'Sergeant'),
    (6, 'CPL', 6, 'Corporal'),
    (7, 'LCPL', 7, 'Lance Corporal'),
    (8, 'CDT', 8, 'Cadet');
```

```
-- 2.2 Load Certificates Reference (dim_certificate)
```

```
INSERT INTO dim_certificate (certificate_id, certificate_name, certificate_level, requirements,
typical_year)
```

```
VALUES
    (1, 'A', 1, 'Advanced Certificate - 3rd year completion with merit', 3),
    (2, 'B', 2, 'Intermediate Certificate - 2nd year completion', 2),
    (3, 'C', 3, 'Basic Certificate - 1st year completion', 1);
```

```
-- 2.3 Load Camps Reference (dim_camp)
```

```
INSERT INTO dim_camp (camp_id, camp_type, camp_description, is_mandatory, typical_duration_days)
VALUES
```

```
    (1, 'CATC', 'Combined Annual Training Camp', 1, 14),
    (2, 'NIC', 'National Integration Camp', 0, 12),
    (3, 'TSC', 'Thal Sainik Camp', 0, 12),
    (4, 'Army Attachment', 'Army Attachment Camp', 0, 15),
    (5, 'ALC', 'Advanced Leadership Camp', 0, 12),
    (6, 'RDC', 'Republic Day Camp', 0, 30),
    (7, 'EBSB', 'Ek Bharat Shreshtha Bharat', 0, 10);
```

```
-- =====
-- STEP 3: GENERATE CALENDAR DIMENSION (dim_date)
-- Recursive CTE to generate dates from 2020-01-01 to 2030-12-31
-- =====
```

```
SET SESSION cte_max_recursion_depth = 10000;
```

```
INSERT INTO dim_date (date_id, calendar_date, day_of_week, day_name, week_of_year, month, month_name,
quarter, year, is_weekend)
```

```
WITH RECURSIVE seq AS (
    SELECT '2020-01-01' AS dt
    UNION ALL
    SELECT dt + INTERVAL 1 DAY FROM seq WHERE dt + INTERVAL 1 DAY <= '2030-12-31'
)
SELECT
    CAST(DATE_FORMAT(dt, '%Y%m%d') AS UNSIGNED) AS date_id,
    dt AS calendar_date,
    DAYOFWEEK(dt) AS day_of_week,
    DAYNAME(dt) AS day_name,
    WEEK(dt) AS week_of_year,
```

```

    MONTH(dt) AS month,
    MONTHNAME(dt) AS month_name,
    QUARTER(dt) AS quarter,
    YEAR(dt) AS year,
    CASE WHEN DAYOFWEEK(dt) IN (1, 7) THEN 1 ELSE 0 END AS is_weekend
FROM seq;

```

```

-- =====
-- STEP 4: POPULATE CADET MASTER DIMENSION (dim_cadet)
-- Combines registration, unit details, and calculated status info
-- =====
INSERT INTO dim_cadet (
    dli_number, full_name, gender, date_of_birth, age_at_enrollment,
    aadhaar_number, email, mobile_number, alternate_mobile_number,
    city, state, pincode, father_name, mother_name, college, course,
    college_roll_number, unit, directorate, group_name, squadron,
    cadet_year, joining_date, enrollment_status, current_rank,
    latest_rank_date, b_certificate_status, c_certificate_status,
    is_active
)
SELECT
    r.dli_number,
    r.full_name,
    r.gender,
    r.date_of_birth,
    -- Simple age calculation since dates are standardized
    CASE
        WHEN r.date_of_birth IS NOT NULL AND nu.joining_date IS NOT NULL
            THEN YEAR(nu.joining_date) - YEAR(r.date_of_birth)
        WHEN r.date_of_birth IS NOT NULL
            THEN YEAR(CURDATE()) - YEAR(r.date_of_birth)
        ELSE NULL
    END AS age_at_enrollment,
    r.aadhaar_number,
    r.email,
    r.mobile_number,
    r.alternate_mobile_number,
    r.city,
    r.state,
    r.pincode,
    r.father_name,
    r.mother_name,
    r.college,
    r.course,
    r.college_roll_number,
    nu.unit,
    nu.directorate,
    nu.group_name,
    nu.squadron,
    nu.cadet_year,
    nu.joining_date,
    nu.enrollment_status,
    -- Get current rank by pulling latest rank details
    COALESCE(
        (SELECT rank_name

```

```

        FROM cadetos_silver.rank_details rd
        WHERE rd.dli_number = r.dli_number
        ORDER BY rd.assigned_date DESC LIMIT 1),
        'CDT'
    ) AS current_rank,
    -- Get latest rank date
    (SELECT MAX(rd.assigned_date)
     FROM cadetos_silver.rank_details rd
     WHERE rd.dli_number = r.dli_number) AS latest_rank_date,
    -- B Certificate completion
    CASE
        WHEN EXISTS (
            SELECT 1 FROM cadetos_silver.certificate_results cr
            WHERE cr.dli_number = r.dli_number AND cr.certificate_type = 'B'
        ) THEN 'Completed'
        ELSE 'Not Completed'
    END AS b_certificate_status,
    -- C Certificate completion
    CASE
        WHEN EXISTS (
            SELECT 1 FROM cadetos_silver.certificate_results cr
            WHERE cr.dli_number = r.dli_number AND cr.certificate_type = 'C'
        ) THEN 'Completed'
        ELSE 'Not Completed'
    END AS c_certificate_status,
    -- Active enrollment check
    CASE
        WHEN UPPER(nu.enrollment_status) IN ('COMPLETED', 'PASSED OUT', 'WITHDRAWN') THEN 0
        ELSE 1
    END AS is_active
FROM cadetos_silver.registration r
LEFT JOIN cadetos_silver.ncc_unit nu ON r.dli_number = nu.dli_number
WHERE r.dli_number IS NOT NULL;

-- =====
-- STEP 5: POPULATE FACT TABLES
-- =====

-- 5.1 Load Parade Attendance Facts (fct_attendance)
INSERT INTO fct_attendance (
    dli_number, cadet_id, parade_date, parade_date_id, attendance_status, is_late, is_excused
)
SELECT
    a.dli_number,
    dc.cadet_id,
    a.parade_date,
    CAST(DATE_FORMAT(a.parade_date, '%Y%m%d') AS UNSIGNED) AS parade_date_id,
    a.attendance_status,
    CASE WHEN a.late = 'Yes' THEN 1 ELSE 0 END AS is_late,
    CASE WHEN a.excused = 'Yes' THEN 1 ELSE 0 END AS is_excused
FROM cadetos_silver.attendance a
LEFT JOIN dim_cadet dc ON a.dli_number = dc.dli_number
WHERE a.dli_number IS NOT NULL;

-- 5.2 Load Camp Participation Facts (fct_camp_participation)

```

```
INSERT INTO fct_camp_participation (
    dli_number, cadet_id, camp_type, camp_location, camp_year, camp_type_id, is_completed
)
SELECT
    c.dli_number,
    dc.cadet_id,
    c.camp_type,
    c.camp_location,
    c.camp_year,
    dc_camp.camp_id,
    1 AS is_completed
FROM cadetos_silver.camp_details c
LEFT JOIN dim_cadet dc ON c.dli_number = dc.dli_number
LEFT JOIN dim_camp dc_camp ON c.camp_type = dc_camp.camp_type
WHERE c.dli_number IS NOT NULL;

-- 5.3 Load Rank Assignment Timeline (fct_rank_assignment)
INSERT INTO fct_rank_assignment (
    dli_number, cadet_id, rank_name, rank_id, assigned_date, assigned_date_id, is_current_rank
)
SELECT
    r.dli_number,
    dc.cadet_id,
    r.rank_name,
    dr.rank_id,
    r.assigned_date,
    CAST(DATE_FORMAT(r.assigned_date, '%Y%m%d') AS UNSIGNED) AS assigned_date_id,
    CASE
        WHEN r.assigned_date = (
            SELECT MAX(rd.assigned_date)
            FROM cadetos_silver.rank_details rd
            WHERE rd.dli_number = r.dli_number
        ) THEN 1
        ELSE 0
    END AS is_current_rank
FROM cadetos_silver.rank_details r
LEFT JOIN dim_cadet dc ON r.dli_number = dc.dli_number
LEFT JOIN dim_rank dr ON r.rank_name = dr.rank_name
WHERE r.dli_number IS NOT NULL;

-- 5.4 Load Certificate Achievements Facts (fct_certificate_achievement)
INSERT INTO fct_certificate_achievement (
    dli_number, cadet_id, certificate_type, certificate_id, grade, passing_year, is_passed
)
SELECT
    c.dli_number,
    dc.cadet_id,
    c.certificate_type,
    dc_cert.certificate_id,
    c.grade,
    c.passing_year,
    CASE WHEN c.grade NOT IN ('F', 'D') THEN 1 ELSE 0 END AS is_passed
FROM cadetos_silver.certificate_results c
LEFT JOIN dim_cadet dc ON c.dli_number = dc.dli_number
LEFT JOIN dim_certificate dc_cert ON c.certificate_type = dc_cert.certificate_name
```

```
WHERE c.dli_number IS NOT NULL;

-- =====
-- STEP 6: UPDATE PRE-CALCULATED METRICS IN dim_cadet
-- =====

UPDATE dim_cadet dc
SET total_camp_count = (
    SELECT COUNT(*)
    FROM fct_camp_participation fcp
    WHERE fcp.cadet_id = dc.cadet_id
);

UPDATE dim_cadet dc
SET total_attendance_records = (
    SELECT COUNT(*)
    FROM fct_attendance fa
    WHERE fa.cadet_id = dc.cadet_id
);

-- =====
-- STEP 7: POPULATE AGGREGATE/SUMMARY TABLES
-- =====

-- 7.1 Populate Cadet Annual Metrics (agg_cadet_annual_metrics)
INSERT INTO agg_cadet_annual_metrics (
    cadet_id, dli_number, year, total_parades, present_count,
    absent_count, leave_count, attendance_percentage, camps_attended,
    ranks_held
)
WITH cadet_annual_attendance AS (
    SELECT
        dc.cadet_id,
        dc.dli_number,
        YEAR(fa.parade_date) AS yr,
        COUNT(DISTINCT fa.parade_date) AS total_parades,
        SUM(CASE WHEN fa.attendance_status = 'Present' THEN 1 ELSE 0 END) AS present_count,
        SUM(CASE WHEN fa.attendance_status = 'Absent' THEN 1 ELSE 0 END) AS absent_count,
        SUM(CASE WHEN fa.attendance_status = 'Leave' THEN 1 ELSE 0 END) AS leave_count
    FROM fct_attendance fa
    JOIN dim_cadet dc ON fa.cadet_id = dc.cadet_id
    GROUP BY dc.cadet_id, dc.dli_number, YEAR(fa.parade_date)
)
SELECT
    caa.cadet_id,
    caa.dli_number,
    caa.yr AS year,
    caa.total_parades,
    caa.present_count,
    caa.absent_count,
    caa.leave_count,
    ROUND(caa.present_count * 100.0 / caa.total_parades, 2) AS attendance_percentage,
    (SELECT COUNT(DISTINCT camp_participation_id)
     FROM fct_camp_participation fcp
     WHERE fcp.cadet_id = caa.cadet_id
     AND fcp.camp_year = caa.yr) AS camps_attended,
```

```

        (SELECT COUNT(DISTINCT rank_assignment_id)
         FROM fct_rank_assignment fra
         WHERE fra.cadet_id = caa.cadet_id
         AND YEAR(fra.assigned_date) = caa.yr) AS ranks_held
FROM cadet_annual_attendance caa;

-- 7.2 Populate Unit Summary (agg_unit_summary)
INSERT INTO agg_unit_summary (
    unit, directorate, squadron, total_cadets, active_cadets,
    completed_cadets, avg_attendance_percentage, total_ranks_assigned
)
WITH cadet_latest_attendance AS (
    -- Get each cadet's latest attendance rate first without grouping
    SELECT
        dc.cadet_id,
        dc.unit,
        dc.directorate,
        dc.squadron,
        dc.is_active,
        COALESCE(
            (SELECT acam.attendance_percentage
             FROM agg_cadet_annual_metrics acam
             WHERE acam.cadet_id = dc.cadet_id
             ORDER BY acam.year DESC LIMIT 1),
            0
        ) AS latest_att_pct
    FROM dim_cadet dc
),
squadron_ranks AS (
    -- Pre-calculate ranks assigned per squadron
    SELECT
        dc.unit,
        dc.squadron,
        COUNT(*) AS total_ranks
    FROM fct_rank_assignment fra
    JOIN dim_cadet dc ON fra.cadet_id = dc.cadet_id
    GROUP BY dc.unit, dc.squadron
)
SELECT
    cla.unit,
    cla.directorate,
    cla.squadron,
    COUNT(DISTINCT cla.cadet_id) AS total_cadets,
    SUM(cla.is_active) AS active_cadets,
    SUM(1 - cla.is_active) AS completed_cadets,
    ROUND(AVG(cla.latest_att_pct), 2) AS avg_attendance_percentage,
    COALESCE(MAX(sr.total_ranks), 0) AS total_ranks_assigned
FROM cadet_latest_attendance cla
LEFT JOIN squadron_ranks sr ON cla.unit = sr.unit AND cla.squadron = sr.squadron
GROUP BY cla.unit, cla.directorate, cla.squadron;

SET SQL_SAFE_UPDATES = 1;

-- =====
-- FINAL VALIDATION SUMMARY

```

```
-- =====
SELECT '=== GOLD LAYER TRANSFORMATION COMPLETE ===' AS status;

SELECT 'DIMENSIONS' AS table_group, 'dim_cadet' AS table_name, COUNT(*) AS count FROM dim_cadet
UNION ALL
SELECT 'DIMENSIONS', 'dim_rank', COUNT(*) FROM dim_rank
UNION ALL
SELECT 'DIMENSIONS', 'dim_certificate', COUNT(*) FROM dim_certificate
UNION ALL
SELECT 'DIMENSIONS', 'dim_camp', COUNT(*) FROM dim_camp
UNION ALL
SELECT 'DIMENSIONS', 'dim_date', COUNT(*) FROM dim_date
UNION ALL
SELECT 'FACTS', 'fct_attendance', COUNT(*) FROM fct_attendance
UNION ALL
SELECT 'FACTS', 'fct_camp_participation', COUNT(*) FROM fct_camp_participation
UNION ALL
SELECT 'FACTS', 'fct_rank_assignment', COUNT(*) FROM fct_rank_assignment
UNION ALL
SELECT 'FACTS', 'fct_certificate_achievement', COUNT(*) FROM fct_certificate_achievement
UNION ALL
SELECT 'SUMMARIES', 'agg_cadet_annual_metrics', COUNT(*) FROM agg_cadet_annual_metrics
UNION ALL
SELECT 'SUMMARIES', 'agg_unit_summary', COUNT(*) FROM agg_unit_summary;
```